

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – FUNCTION CALLING & STRUCTURED OUTPUT TASK

Course Code:	CSA6502	Course Title:	Generative AI and Large Scale Models
CO Assessed:	CO2 – Precision (BL2, BL3)	Assessment:	Assessment Tool 3 – Function Calling & Structured Output Task
Weightage:	40% of CIA	Total Marks:	1 * 100 = 100 (1 Question1)
CO2 Statement:	<i>Apply prompt engineering techniques and utilize pre-trained Large Language Models through modern AI frameworks and APIs to solve real-world problems (BL2, BL3)</i>		
Student Name:	V Rohan	Reg. No.:	192472227
Section / Batch:	CSE - AI	Date:	04 / 08 / 2026

Code of Conduct

I __ V Rohan (192472227) __ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: __ Rohan __

SECTION A — FUNCTION CALLING & STRUCTURED OUTPUT TASK

25. Multi-Function Customer Service Agent

A customer-service assistant should be able to check order status, cancel an order, and create a complaint.

Design three function schemas and explain how the LLM determines which function should be called for different customer requests.

Solution:

1. Scenario

A customer-service assistant for an e-commerce platform must be able to (a) check the status of an existing order, (b) cancel an order, and (c) log a complaint/support ticket when the customer is dissatisfied. The assistant is built on an LLM that has been given a set of callable functions (tools). Given a free-text customer message, the LLM must decide, on its own, which function (if any) applies, extract the correct parameters from the conversation, and only then trigger the backend call. The backend response must come back to the user as a clean, structured, predictable object rather than free-form text.

2. Objective

- Design three function-calling schemas: `check_order_status()`, `cancel_order()`, `create_complaint()`.
- Show the complete pipeline: User Query → LLM (intent + parameter extraction) → Function/Tool Call → Function Execution (backend/mock) → Structured JSON Response → Natural-language reply.
- Explain how the LLM chooses the correct function among three candidates for different customer requests.
- Handle edge cases: missing order ID, ambiguous order reference, invalid/unknown order, and requests that don't map to any function.

3. Function-Calling Schema Design

All three functions are exposed to the LLM in OpenAI-style tool-calling JSON format. Each schema defines the function name, a description the model uses to decide relevance, a parameters object (JSON Schema draft), and which fields are required.

3.1 `check_order_status()`

Purpose: Retrieves the current status, items, and expected delivery date for a single order belonging to the customer. Used whenever the customer wants to know 'where is my order' / 'has it shipped' / delivery-related queries.

```
{
  "type": "function",
  "function": {
    "name": "check_order_status",
    "description": "Retrieve the live status, items and expected delivery date of a customer's order. Call this when the user asks about order status, tracking, delivery date, or shipment.",
    "parameters": {
      "type": "object",
      "properties": {
        "customer_id": { "type": "string", "description": "Unique ID of the logged-in customer" },
```

```

    "order_id": { "type": "string", "description": "Unique order identifier, e.g. ORD10234" }
  },
  "required": ["customer_id", "order_id"]
}
}
}

```

3.2 cancel_order()

Purpose: Cancels an active order that has not yet shipped, and returns refund status. Used when the customer explicitly wants to stop/cancel an order.

```

{
  "type": "function",
  "function": {
    "name": "cancel_order",
    "description": "Cancel a specific, already-placed order and initiate a refund if eligible. Only call this when the user has clearly identified ONE order to cancel and has confirmed the cancellation.",
    "parameters": {
      "type": "object",
      "properties": {
        "customer_id": { "type": "string", "description": "Unique ID of the logged-in customer" },
        "order_id": { "type": "string", "description": "Unique order identifier to cancel" },
        "reason": {
          "type": "string",
          "enum": ["changed_mind", "found_cheaper", "wrong_item", "delivery_delay", "other"],
          "description": "Customer's stated reason for cancellation"
        },
      },
      "confirmed": { "type": "boolean", "description": "True only after the customer has explicitly confirmed the cancellation" }
    },
    "required": ["customer_id", "order_id", "confirmed"]
  }
}

```

3.3 create_complaint()

Purpose: Logs a support/complaint ticket for issues such as damaged items, wrong delivery, or poor service, and returns a ticket ID the customer can track.

```

{
  "type": "function",
  "function": {
    "name": "create_complaint",
    "description": "Create a customer-support complaint ticket. Call this when the user is reporting a problem, dissatisfaction, damaged/missing item, or wants to escalate an issue.",
    "parameters": {
      "type": "object",
      "properties": {
        "customer_id": { "type": "string", "description": "Unique ID of the logged-in customer" },
        "order_id": { "type": "string", "description": "Related order ID, if the complaint concerns a specific order" },
        "issue_category": {
          "type": "string",
          "enum": ["damaged_item", "wrong_item", "missing_item", "late_delivery", "billing", "other"],
          "description": "Category of the reported issue"
        },
      },
      "description": { "type": "string", "description": "Free-text description of the issue in the customer's own words" }
    },
    "required": ["customer_id", "issue_category", "description"]
  }
}

```

```
}  
}
```

4. Structured JSON Output Schemas (Function Responses)

Each function returns a fixed-shape JSON object. This is what the backend produces after execution, which the LLM then reads and turns into a natural reply.

4.1 check_order_status – response schema

```
{  
  "order_id": "ORD10234",  
  "status": "Shipped | Processing | Out for Delivery | Delivered | Cancelled | Not Found",  
  "items": [  
    { "name": "Wireless Mouse", "quantity": 1 }  
  ],  
  "expected_delivery": "2026-08-08",  
  "found": true  
}
```

4.2 cancel_order – response schema

```
{  
  "order_id": "ORD10234",  
  "cancellation_status": "Success | Failed | Already Shipped | Not Found",  
  "refund_status": "Initiated | Not Applicable | N/A",  
  "cancelled_at": "2026-08-04T14:32:00Z",  
  "message": "string"  
}
```

4.3 create_complaint – response schema

```
{  
  "ticket_id": "TCK58231",  
  "status": "Open",  
  "priority": "Low | Medium | High",  
  "issue_category": "damaged_item",  
  "created_at": "2026-08-04T14:35:00Z"  
}
```

5. End-to-End Workflow: User → LLM → Function → Response

The pipeline follows five steps for every customer message:

- 1. User Query – customer types a free-text message.
- 2. LLM Reasoning – the model reads the tool list, matches intent to the closest function description, and extracts parameters from the message and prior conversation/session (customer_id comes from the logged-in session, not typed by the user).
- 3. Function/Tool Call – if all required parameters are present, the LLM emits a structured tool_call with the function name and arguments. If required parameters are missing or the request is ambiguous, the LLM does NOT call the function — it asks a clarifying question instead.
- 4. Function Execution – the application code intercepts the tool_call, executes the real (or mocked) backend logic, and returns the structured JSON response.

- 5. Structured Response – the JSON result is fed back to the LLM as a tool result; the LLM converts it into a short, natural-language reply for the customer, while the JSON itself can also be logged/displayed for auditing.

5.1 Routing logic – how the LLM picks the right function

The three tool descriptions are written to be mutually exclusive on intent keywords, so the model's function selection is a semantic match, not a lookup table:

Customer message	Function selected	Why
"Where is my order ORD10234?"	check_order_status	Intent = tracking/status, single order clearly identified.
"Cancel order ORD10234, I found it cheaper elsewhere."	cancel_order	Explicit cancel verb + order ID + reason present.
"My package arrived damaged, this is unacceptable."	create_complaint	Intent = reporting a problem, no cancel/status verb.
"Cancel my order." (2 active orders on file)	none yet — clarify	order_id required but ambiguous; LLM must disambiguate first.

6. Implementation (Python, OpenAI-style function calling)

```
import json
from datetime import datetime, timezone

# ----- Mock backend "database" -----
ORDERS = {
    "ORD10234": {"status": "Shipped", "items": [{"name": "Wireless Mouse", "quantity": 1}],
                "expected_delivery": "2026-08-08", "shipped": True},
    "ORD10555": {"status": "Processing", "items": [{"name": "Bluetooth Speaker", "quantity": 2}],
                "expected_delivery": "2026-08-10", "shipped": False},
}

# ----- Function implementations -----
def check_order_status(customer_id, order_id):
    order = ORDERS.get(order_id)
    if not order:
        return {"order_id": order_id, "status": "Not Found", "items": [],
                "expected_delivery": None, "found": False}
    return {"order_id": order_id, "status": order["status"], "items": order["items"],
            "expected_delivery": order["expected_delivery"], "found": True}

def cancel_order(customer_id, order_id, confirmed, reason=None):
    order = ORDERS.get(order_id)
    if not order:
        return {"order_id": order_id, "cancellation_status": "Not Found",
                "refund_status": "N/A", "cancelled_at": None,
                "message": "No such order for this customer."}
    if not confirmed:
        return {"order_id": order_id, "cancellation_status": "Failed",
                "refund_status": "N/A", "cancelled_at": None,
                "message": "Cancellation requires explicit customer confirmation."}
    if order["shipped"]:
        return {"order_id": order_id, "cancellation_status": "Already Shipped",
                "refund_status": "Not Applicable", "cancelled_at": None,
                "message": "Order has already shipped and cannot be cancelled online."}
```

```

order["status"] = "Cancelled"
return {"order_id": order_id, "cancellation_status": "Success",
        "refund_status": "Initiated",
        "cancelled_at": datetime.now(timezone.utc).isoformat(),
        "message": "Order cancelled and refund initiated."}

def create_complaint(customer_id, issue_category, description, order_id=None):
    ticket_id = f"TCK{abs(hash((customer_id, description))) % 100000}"
    priority = "High" if issue_category in ("damaged_item", "missing_item") else "Medium"
    return {"ticket_id": ticket_id, "status": "Open", "priority": priority,
            "issue_category": issue_category,
            "created_at": datetime.now(timezone.utc).isoformat()}

FUNCTIONS = {
    "check_order_status": check_order_status,
    "cancel_order": cancel_order,
    "create_complaint": create_complaint,
}

def execute_tool_call(name, arguments: dict):
    """Runs after the LLM emits a structured tool_call {name, arguments}."""
    if name not in FUNCTIONS:
        raise ValueError(f"Unknown function: {name}")
    return FUNCTIONS[name](**arguments)

```

7. Test Cases and Results

Five test cases are used, covering normal flow, multi-turn parameter completion, and edge cases (missing parameter, ambiguous input, invalid data).

Test Case 1 – Normal status query

Input: "Hi, can you check the status of order ORD10234?"

LLM tool call: `check_order_status(customer_id="CUST001", order_id="ORD10234")`

```
{ "order_id": "ORD10234", "status": "Shipped", "items": [{ "name": "Wireless Mouse", "quantity": 1 }],
  "expected_delivery": "2026-08-08", "found": true }
```

Final reply: Your order ORD10234 has shipped and is expected to arrive by 8 Aug 2026.

Test Case 2 – Cancel with confirmation

Input: "Please cancel order ORD10555, I found it cheaper elsewhere." → assistant asks to confirm → "Yes, cancel it."

LLM tool call: `cancel_order(customer_id="CUST001", order_id="ORD10555", reason="found_cheaper", confirmed=true)`

```
{ "order_id": "ORD10555", "cancellation_status": "Success", "refund_status": "Initiated",
  "cancelled_at": "2026-08-04T14:32:00Z", "message": "Order cancelled and refund initiated." }
```

Test Case 3 – Complaint logging

Input: "The speaker I received in ORD10555 was broken out of the box."

LLM tool call: `create_complaint(customer_id="CUST001", order_id="ORD10555", issue_category="damaged_item", description="Speaker arrived broken out of the box")`

```
{ "ticket_id": "TCK48213", "status": "Open", "priority": "High",
  "issue_category": "damaged_item", "created_at": "2026-08-04T14:35:00Z" }
```

Test Case 4 (Edge case) – Missing required parameter

Input: "I want to cancel my order." (order_id not stated, confirmed not given)

LLM behaviour: No tool call is emitted. The model asks: "Sure — which order would you like to cancel? Could you share the order ID?" Only after the customer supplies order_id AND confirms does cancel_order() actually fire. This prevents an invalid/incomplete function call.

Test Case 5 (Edge case) – Ambiguous reference with multiple active orders

Input: "Cancel my order." (customer has ORD10234 and ORD10555 both active)

LLM behaviour: The model does not guess. It lists the customer's active orders and asks the user to pick one, e.g. "You have two active orders: ORD10234 (Shipped) and ORD10555 (Processing). Which one should I cancel?" This avoids the failure mode of cancelling the wrong order.

Test Case 6 (Edge case) – Invalid / unknown order ID

Input: "What's the status of order ORD99999?"

LLM tool call: check_order_status(customer_id="CUST001", order_id="ORD99999")

```
{ "order_id": "ORD99999", "status": "Not Found", "items": [], "expected_delivery": null, "found": false }
```

Final reply: I couldn't find an order with ID ORD99999 under your account. Could you double-check the order number?

8. Schema Documentation

Function	Parameter	Type	Required	Purpose
check_order_status	customer_id / order_id	string	Yes / Yes	Identify caller and the exact order to look up
cancel_order	customer_id / order_id	string	Yes / Yes	Identify caller and the exact order to cancel
cancel_order	reason	enum(string)	No	Captures cancellation reason for analytics
cancel_order	confirmed	boolean	Yes	Safety gate — blocks execution until customer explicitly confirms
create_complaint	customer_id / description	string	Yes / Yes	Identify caller and capture the issue in the customer's words
create_complaint	order_id	string	No	Links ticket to an order when relevant (not all

Function	Parameter	Type	Required	Purpose
				complaints are order-specific)
create_complaint	issue_category	enum(string)	Yes	Enables automatic routing/priority assignment

9. Design Justification

Function calling and structured output improve this assistant in three concrete ways:

- Reliability – because the schema constrains the model's output to a fixed set of fields and types (e.g. `issue_category` can only be one of six enum values), downstream code never has to parse free text or guess intent; a malformed call simply fails validation instead of silently corrupting an order.
- Consistency – every `check_order_status` call returns the exact same JSON shape whether the order exists or not (found: true/false), so the front-end and logging systems can be written once against a stable contract rather than branching on unpredictable text.
- Usability & safety – required fields like `confirmed` on `cancel_order` force the model to pause and get explicit consent before an irreversible action, and the model is instructed to withhold the tool call entirely when information is missing or ambiguous (Test Cases 4 and 5), which prevents wrong-order cancellations and other costly mistakes that a purely conversational, unconstrained LLM reply could cause.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Schema Design	30%	Correctly designs a function-calling schema and structured (JSON) output format	Mostly correct schema with minor issues	Schema has significant errors	Schema is fundamentally incorrect or missing
Output Validity	25%	LLM reliably produces valid structured output matching the schema	Mostly valid output with occasional errors	Output is frequently invalid or inconsistent	Output rarely or never matches the schema
Edge-Case Handling	20%	Handles edge cases (missing fields, ambiguous input) gracefully	Handles common cases well	Handles only the simplest cases	Fails on basic cases
Function-Call Flow Demo	15%	Clear demo of the function-call to structured-response flow	Demo covers the main flow	Demo is incomplete	No functional demo presented
Schema Documentation	10%	Well-documented schema design rationale	Adequate documentation	Minimal documentation	No documentation provided

Marks Evaluation:

Question No.	Schema Design(30%)	Output Validity (25%)	Edge-Case Handling (20%)	Function-Call Flow Demo(15%)	Schema Documentation (10%)	Total Marks (100)
Q1						
Overall Total (100)						
Faculty In-charge		Course Coordinator		Head of Department		
Signature: _____		Signature: _____		Signature: _____		
Date: _____		Date: _____		Date: _____		